

Efficient Verification Framework of LTL via Transition Certificates

YUHENG FAN, Zhejiang Sci-Tech University, China

ZENAN QIAN, Zhejiang Sci-Tech University, China

WANG LIN, Zhejiang Sci-Tech University, China

Verifying Linear Temporal Logic (LTL) specifications for discrete-time dynamical systems with continuous state spaces is critical for safety-critical applications. The existing state-triplet method and closure certificate often face a trade-off between conservatism and high computational cost. To overcome the limitations, we propose a novel framework to verify LTL specifications via proof certificates. The key insight of our approach is to decompose LTL verification into two types of certificates: transition persistence certificates, which guarantee accepting transitions take finitely often, and transition safety certificates, which render accepting transitions unreachable. This design not only reduces the search space significantly compared to closure certificates but also mitigates conservatism inherent in state-triplet methods. We introduce automated synthesis techniques based on polynomial and neural network templates, balancing expressiveness and computational efficiency. Experimental evaluations on benchmark case studies, including the Kuramoto oscillator and a two-room temperature model, show that transition certificates achieve significant speedup over baseline methods while providing formal guarantees, highlighting their practicality for LTL verification tasks.

ACM Reference Format:

Yuheng Fan, Zenan Qian, and Wang Lin. 2018. Efficient Verification Framework of LTL via Transition Certificates. *Proc. ACM Meas. Anal. Comput. Syst.* 37, 4, Article 111 (August 2018), 20 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Cyber-Physical Systems (CPS) [?] integrate computational and physical capabilities and are now common in safety-critical applications like autonomous driving and medical devices [?]. For such systems, any mistake of them may result in catastrophic consequences. Linear Temporal Logic (LTL) is a formal language used to define the complex rules for safe operation, such as requiring a timely response or forbidding unsafe conditions [?]. Therefore, the ability to verify that a CPS follows its LTL specification is essential for building trust and ensuring safety in these systems.

Although important, formally verifying LTL specifications for CPS is a significant challenge. The main difficulty arises from the need to handle the infinite-horizon nature of LTL properties [?] together with the infinite state spaces and complex dynamics of physical systems. To address this, researchers have developed certificate-based methods, including the state-triplet method [?] and closure certificates [?]. The state-triplet method provides an automata-theoretic approach for verifying LTL specifications of dynamical systems. This technique constructs barrier certificates that prevent transitions to accepting states in the automaton representing the negated specification. This approach can be conservative as it requires blocking all potential paths to accepting states individually. The method's strength lies in its direct connection to automata structure, but it may need numerous certificates for complex specifications. Closure certificates offer a generalized framework by operating on pairs of

Authors' Contact Information: Yuheng Fan, Zhejiang Sci-Tech University, Hangzhou, Zhejiang, China; Zenan Qian, Zhejiang Sci-Tech University, Hangzhou, Zhejiang, China; Wang Lin, linwang@zstu.edu.cn, Zhejiang Sci-Tech University, Hangzhou, Zhejiang, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, or post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2476-1249/2018/8-ART111

<https://doi.org/XXXXXXX.XXXXXXX>

system states through transition invariants. This approach establishes certificates that enforce decrease conditions along trajectories, ensuring accepting states are visited only finitely often. The method reduces conservatism by allowing more flexible certificate structures and naturally handles liveness specifications. While these methods are significant, they each have trade-offs. The state-triplet method can be conservative and closure certificates have low synthesis efficiency because of the large search space they must explore. These limitations show a clear need for a new framework that can reduce conservatism without a high computational cost.

To address this gap, we propose a new framework to verify LTL specifications via proof certificates. Our approach shifts the focus of the LTL verification problem to the transitions of the Nondeterministic Büchi Automaton (NBA) [?]. In our framework, verification is achieved by finding two types of complementary certificates. Transition safety certificates prove that certain accepting transitions never occur, while transition persistence certificates ensure that other accepting transitions happen only a finite number of times. By combining them, we can formally guarantee that no accepting run exists. To find these certificates, we developed an automated synthesis method based on the Counterexample-Guided Inductive Synthesis (CEGIS) framework. This method supports both polynomial templates for efficiency and neural network templates for expressiveness. The main contributions of this paper are summarized as follows:

- We propose a new LTL verification framework for discrete-time dynamical systems with continuous state spaces, which employs transition certificates to reduce the search space, thereby achieving higher computational efficiency compared with closure certificates.
- We introduce automated synthesis methods for transition certificates based on polynomial and neural network templates. These templates provide a balance between expressiveness and computational efficiency, enabling efficient certificate generation for diverse system classes, from linear to nonlinear dynamics.
- We demonstrate the effectiveness of our certificates through case studies, including the Kuramoto oscillator and a two-room temperature model. Experimental results show that transition certificates achieve significant speedups, highlighting their practicality for LTL specifications.

The organization of this paper is as follows: Section 2 covers preliminary knowledge. Section 3 introduces our proof certificates for LTL verification and compares with other methods. Section 4 presents synthesis methods. Section 5 demonstrates the effectiveness through case studies. Section 6 concludes the paper.

Related work: Verification of temporal logic properties via proof certificates has seen significant advancements. To validate discrete-time systems against LTL specifications, Wongpiromsarn et al. [?] developed a state-triplet method that uses barrier certificates to block all simple paths from initial states to accepting states in the automaton, requiring enumeration of state triplets (q_m, q'_m, q''_m) along these paths. Murali et al. [?] introduced closure certificates for persistence verification, which establish disjunctively well-founded transition invariants [?] for dynamical systems, and further extended this notion to control closure certificates for ω -regular specifications [?]. Additionally, substantial research has addressed the verification of specific temporal behaviors such as reachability, safety, and avoidance [? ? ? ?]. Some adopt neural networks as templates [? ? ?] to synthesize certificates for verifying temporal properties, including the simultaneous representation of ranking functions and inductive invariants [?] and the verification of memoryful neural agents against LTL specifications [?].

The abstraction-based approaches [? ? ? ? ? ?] lift the CPS model from continuous domain to discrete domain by computing a finite abstraction, e.g., finite transition systems and Markov decision processes (MDPs). We note that abstraction-based techniques have been used in the verification and synthesis of continuous-space dynamical systems against LTL properties such as the results in [? ? ? ?]. These results rely on building a finite state abstraction and then making use of model checking and synthesis techniques [? ?]. In general, the abstraction-based approaches are computationally demanding and suffer from the curse of dimensionality. Abstraction-free methods, such as proof certificates, can alleviate the computational burden to some extent.

2 Preliminaries

2.1 Notation

We use $\mathbb{N}$ and $\mathbb{R}$ to represent the set of natural numbers and the set of real numbers, respectively. We define:

$$\mathbb{N}_{\sim i} = \{n \in \mathbb{N} \mid n \sim i\}, \quad \text{where } i \in \mathbb{N}, \sim \in \{\geq, \leq, >, <\},$$

$$\mathbb{R}_{\sim i} = \{r \in \mathbb{R} \mid r \sim i\}, \quad \text{where } i \in \mathbb{R}, \sim \in \{\geq, \leq, >, <\}.$$

Given a set S and a set S_1 , we define an indicator function:

$$I_S(x) = \begin{cases} 1, & \text{if } x \in S \\ 0, & \text{if } x \notin S. \end{cases}$$

$|S|$ denotes the cardinality of S and $S \setminus S_1$ denotes $\{x \in S \mid x \notin S_1\}$.

Given a sequence $s = \langle s_0, s_1, \dots \rangle$, where $s_i \in S$ for $i \in \mathbb{N}$:

- $s[i]$ denotes s_i ;
- $s[i : j]$ denotes $\langle s_i, \dots, s_j \rangle$ for $j \geq i$;
- $s[i : \infty]$ denotes $\langle s_i, s_{i+1}, \dots \rangle$.

We say s visits S if there exists $i \in \mathbb{N}$ such that $s_i \in S$. $\text{Inf}(s)$ denotes the set of elements that appear in s infinitely often. S^ω denotes the set of infinite sequences over S , i.e., $S^\omega = \{s \mid s = \langle s_0, s_1, \dots \rangle \text{ and } s_i \in S \text{ for } i \in \mathbb{N}\}$. A function $f : S \rightarrow \mathbb{R}$ is bounded if and only if there exist $a, b \in \mathbb{R}$ such that $a \leq f(x) \leq b$ for all $x \in S$.

2.2 Discrete-time Dynamical System

A discrete-time dynamical system is defined as $\mathfrak{S} = (X, X_0, f)$, where $X \subseteq \mathbb{R}^n$ represents the state set, $X_0 \subseteq X$ represents a set of initial states, and $f : X \rightarrow X$ represents the transition function. The evolution of the system state is described as follows:

$$x_{t+1} = f(x_t).$$

An infinite sequence of states $\langle x_0, x_1, x_2, \dots \rangle$ is called a *trajectory* of the system if $x_0 \in X_0$ and $x_{i+1} = f(x_i)$ for $i \in \mathbb{N}$. We use $\mathfrak{T}(\mathfrak{S})$ to represent all *trajectories* on $\mathfrak{S}$. The *predecessor* of x_{i+1} is x_i for $i \in \mathbb{N}$. In a given *trajectory*, x_{i+1} has only one *predecessor*. We use the notation x_i^{pre} to denote the *predecessor* of x_i in a *trajectory*.

2.3 Specification

While this paper focuses on the verification of LTL properties for discrete-time dynamical systems, we also cover foundational concepts such as safety, persistence, and Nondeterministic Büchi Automata (NBA), which are essential for establishing our framework.

Safety: A system $\mathfrak{S} = (X, X_0, f)$ is *safe* with respect to unsafe states $X_u \subseteq X$ if for any *trajectory* $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$, we have $x_i \notin X_u$ for all $i \in \mathbb{N}$.

Persistence: A system $\mathfrak{S} = (X, X_0, f)$ is *persistent* with respect to states $X_{VF} \subseteq X$ if for any *trajectory* $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$, elements of X_{VF} appear finitely often in τ .

Linear Temporal Logic (LTL) [?]: LTL formulae are defined over a set of atomic propositions AP . A labelling function $\mathfrak{L} : X \rightarrow 2^{AP}$ maps system states to subsets of AP . Given a *trajectory* $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$, we denote $\mathfrak{L}(\tau) = \langle \mathfrak{L}(x_0), \mathfrak{L}(x_1), \dots \rangle$ as the corresponding *word*.

The syntax of LTL is:

$$\phi := \top \mid a \mid \neg\phi \mid \phi \wedge \phi \mid X\phi \mid \phi U \phi,$$

where $a \in AP$ is an atomic proposition. Temporal operators finally (F) and always (G) are derived from next (X) and until (U).

Given a *word* $w = \mathfrak{L}(\tau)$ and an LTL formula ϕ , the semantics is defined inductively:

- $w \models \top$ always holds,
- $w \models a$ iff $a \in w[0]$,
- $w \models \phi_1 \wedge \phi_2$ iff $w \models \phi_1$ and $w \models \phi_2$,
- $w \models \neg\phi_1$ iff $w \not\models \phi_1$,
- $w \models X\phi_1$ iff $w[1 : \infty] \models \phi_1$,
- $w \models \phi_1 U \phi_2$ iff there exists $j \in \mathbb{N}$ such that $w[j : \infty] \models \phi_2$ and for all $i \in \mathbb{N}_{<j}$, $w[i : \infty] \models \phi_1$.

We say $\mathfrak{S} \models_{\mathfrak{L}} \phi$ if for any $\tau \in \mathfrak{T}(\mathfrak{S})$, we have $\mathfrak{L}(\tau) \models \phi$.

Nondeterministic Büchi Automaton (NBA) [?]: An NBA is a tuple $(\Sigma, Q, q_0, \delta, Acc)$, where $\Sigma = 2^{AP}$ is the alphabet, Q is a finite set of states, $q_0 \in Q$ is the initial state, $\delta \subseteq Q \times \Sigma \times Q$ is the transition relation, and $Acc \subseteq \delta$ is the set of *accepting transitions*. Given a word $w = \langle w_0, w_1, \dots \rangle \in \Sigma^\omega$, a **run** on w is a sequence $r = \langle (r_0, w_0, r_1), (r_1, w_1, r_2), \dots \rangle$ where $r_0 = q_0$ and $(r_i, w_i, r_{i+1}) \in \delta$ for all $i \in \mathbb{N}$. The *transition-based accepting condition* is that an NBA accepts w if there exists a run where at least one accepting transition occurs infinitely often, i.e., $Inf(r) \cap Acc \neq \emptyset$.

For an NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, Acc)$, we define:

$$Acc^{pair} = \{(p, q) \mid \exists \sigma \in \Sigma. (p, \sigma, q) \in Acc\}, \quad (1)$$

$$Acc^{start} = \{p \mid \exists \sigma \in \Sigma, q \in Q. (p, \sigma, q) \in Acc\}, \quad (2)$$

$$Acc^{k,l} = \{(q_k, \sigma, q_l) \mid (q_k, \sigma, q_l) \in Acc\}. \quad (3)$$

We denote $\mathfrak{L}(\mathfrak{S}) = \{\mathfrak{L}(\tau) \mid \tau \in \mathfrak{T}(\mathfrak{S})\}$ as the *language* of $\mathfrak{S}$, and $\mathfrak{L}(\mathcal{A})$ as the language accepted by $\mathcal{A}$. To verify $\mathfrak{S} \models_{\mathfrak{L}} \phi$, we construct an NBA $\mathcal{A}$ for $\neg\phi$ and show $\mathfrak{L}(\mathfrak{S}) \cap \mathfrak{L}(\mathcal{A}) = \emptyset$.

2.4 Proof Certificates

Barrier certificates [?] and closure certificates [?] are used to LTL properties [? ?], respectively. We provide their definitions and related conclusions from prior work.

Definition 2.1 (barrier certificate [?]). Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ and $\mathcal{X}_u \subseteq \mathcal{X}$, a bounded function $B(x) : \mathcal{X} \rightarrow \mathbb{R}$ is a *barrier certificate* with respect to $\mathcal{X}_u$ such that $x_0 \in \mathcal{X}_0$, $x_u \in \mathcal{X}_u$, $x \in \mathcal{X}$ and $x' = f(x)$. We have:

$$B(x_0) \geq 0, \quad (4)$$

$$B(x_u) < 0, \quad (5)$$

$$B(x) \geq 0 \Rightarrow B(x') \geq 0. \quad (6)$$

THEOREM 2.2 (SAFETY [?]). *The existence of a barrier certificate $B(x)$ implies that the system $\mathfrak{S}$ is safe with respect to $\mathcal{X}_u$.*

The state-triplet approach[?] uses barrier certificates to verify LTL properties. Given an LTL specification ϕ , it proves that accepting states are unreachable in the NBA for $\neg\phi$ by finding barrier certificates to ensure that runs on $\mathfrak{L}(\mathfrak{S})$ will never visit accepting states, verifying $\mathfrak{S} \models_{\mathfrak{L}} \phi$. The state-triplet method need to find barrier certificates for each simple path, a certificate corresponding to a specific state triplet to individually block a simple path. The state-triplet method introduces conservatism: it cannot be applied for systems that satisfy the LTL specification because they cannot be proven safe under this restrictive constraints.

Definition 2.3 (closure certificate [?]). Consider a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a set of atomic propositions AP , a labelling function $\mathfrak{L} : \mathcal{X} \rightarrow 2^{AP}$ and an NBA $\mathcal{A} = (2^{AP}, Q, q_0, \delta, Acc)$ represent $\neg\phi$. A bounded function $T : \mathcal{X} \times Q \times \mathcal{X} \times Q \rightarrow \mathbb{R}$ is a closure certificate for ϕ if there exists a value $\epsilon \in \mathbb{R}_{>0}$ such that for all states

$x, y, y' \in \mathcal{X}$, $x' = f(x)$, states $q_i, q_j \in \mathcal{Q}$, and $q'_i \in \delta(q_i, \mathfrak{L}(x))$, we have:

$$T((x, q_i), (x', q'_i)) \geq 0, \quad (7)$$

$$T((x', q'_i), (y, q_j)) \geq 0 \Rightarrow T((x, q_i), (y, q_j)) \geq 0. \quad (8)$$

and for all states $x_0 \in \mathcal{X}_0$, and $\ell, \ell' \in \text{Acc}$, we have:

$$\begin{aligned} T((x_0, q_0), (y, \ell)) \geq 0 \wedge T((y, \ell), (y', \ell')) \geq 0 \\ \Rightarrow T((x_0, q_0), (y', \ell')) \leq T((x_0, q_0), (y, \ell)) - \epsilon. \end{aligned} \quad (9)$$

THEOREM 2.4 (LTL VERIFICATION [?]). Consider a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a set of atomic propositions AP , a labelling function $\mathfrak{L} : \mathcal{X} \rightarrow 2^{AP}$ and an LTL formula ϕ . Let an NBA $\mathcal{A} = (2^{AP}, \mathcal{Q}, q_0, \delta, \text{Acc})$ represent $\neg\phi$. The existence of a closure certificate satisfying conditions (7)-(9) implies that $\mathfrak{S} \models_{\mathfrak{L}} \phi$.

Closure certificates are defined by characterizing an over-approximation of the transition closure along with a decrease condition, ensuring that transitions in certain regions are visited only finitely often, and are used for LTL verification. Closure certificates (2.3) are defined on $\mathcal{X} \times \mathcal{Q} \times \mathcal{X} \times \mathcal{Q}$, creating high-dimensional search spaces with heavy computational burdens.

2.5 Problem Statement

Problem. Given a discrete-time dynamical system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a set of atomic propositions AP , a labeling function $\mathfrak{L} : \mathcal{X} \rightarrow 2^{AP}$, and an LTL formula ϕ , verify that $\mathfrak{S} \models_{\mathfrak{L}} \phi$ holds.

In this paper, we propose a new approach to address the LTL verification problem.

3 Transition Certificates

In this section, we propose a new type of proof certificates, called transition certificates, to verify that a system $\mathfrak{S}$ satisfies an LTL formula ϕ . The core idea of transition certificates is to reduce the verification problem to analyzing the behavior of accepting transitions in the NBA $\mathcal{A}$ for $\neg\phi$. By the definition of the transition-based acceptance condition, a word is accepted by $\mathcal{A}$ if and only if accepting transitions occur infinitely often along one of its runs. Therefore, to prove $\mathfrak{S} \models_{\mathfrak{L}} \phi$, it suffices to show that for all trajectories of $\mathfrak{S}$, every accepting transition in $\mathcal{A}$ occurs at most finitely many times. We introduce two types of transition certificates: *transition safety certificates* and *transition persistence certificates*. The two certificates address the problem from fundamentally different angles and concern the set $\text{Acc}^{\text{start}}$, which comprises the starting states of all accepting transitions in $\mathcal{A}$ (cf. equation 2).

- A *transition safety certificate* proves that the system's trajectories can never reach $\text{Acc}^{\text{start}}$, thereby guaranteeing that related accepting transitions **never occur**.
- A *transition persistence certificate* allows the system to reach $\text{Acc}^{\text{start}}$ but proves that related accepting transitions can only be taken **finitely often**.

3.1 Transition Safety Certificate

The objective of a transition safety certificate is to prevent any accepting transition from occurring. To achieve this, we verify that accepting transitions never occur by showing that $\text{Acc}^{\text{start}}$ is unreachable in the product of the system and the automaton.

Definition 3.1 (transition safety certificate). Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a labelling function $\mathfrak{L} : \mathcal{X} \rightarrow 2^{AP}$ and an NBA $\mathcal{A} = (2^{AP}, \mathcal{Q}, q_0, \delta, \text{Acc})$. Construct the product system $\mathfrak{S} \times \mathcal{A} = (\mathcal{Y}, \mathcal{Y}_0, F)$, where $\mathcal{Y} = \{(x, q) \mid x \in \mathcal{X}, q \in \mathcal{Q}\}$, $\mathcal{Y}_0 = \{(x_0, q_0) \mid x_0 \in \mathcal{X}_0\}$, and $F(x, q) = \{(x', q') \mid x' = f(x) \wedge (q, \mathfrak{L}(x), q') \in \delta\}$. For $q_k \in \text{Acc}^{\text{start}}$,

define the unsafe set $\mathcal{Y}_u = \{(x, q_k) \mid x \in \mathcal{X}\}$. A bounded function $B_k(x, q) : \mathcal{X} \times \mathcal{Q} \rightarrow \mathbb{R}$ is a *transition safety certificate* with respect to $q_k \in \text{Acc}^{\text{start}}$ if for all $(x, q) \in \mathcal{Y}$, $(x_0, q_0) \in \mathcal{Y}_0$, $(x', q') \in F(x, q)$, we have:

$$B_k(x_0, q_0) \geq 0, \quad (10)$$

$$B_k(x, q_k) < 0, \quad (11)$$

$$B_k(x, q) \geq 0 \Rightarrow B_k(x', q') \geq 0. \quad (12)$$

Intuitively, condition (10) guarantees that the certificate value is non-negative at the initial state of the product system; condition (11) ensures that the certificate becomes negative precisely on the unsafe set $\mathcal{Y}_u^k$ associated with q_k , thereby marking states where accepting transitions could originate; condition (12) implies that once the certificate value is non-negative at a state, it remains non-negative along any trajectory of the product system, thus preserving the safety invariant.

Collectively, these conditions ensure that the certificate B_k guarantees q_k is unreachable, preventing all accepting transitions from q_k . The following lemma formalizes this guarantee. Its proof is provided in Appendix A.

LEMMA 3.2. *If a transition safety certificate B_k with respect to $q_k \in \text{Acc}^{\text{start}}$ can be found, then all runs of words in $\mathcal{L}(\mathfrak{S})$ never visit state q_k , thereby preventing all accepting transitions starting from q_k .*

The next theorem follows directly from applying the above lemma to every state in $\text{Acc}^{\text{start}}$. If a transition safety certificate can be found for each state in $\text{Acc}^{\text{start}}$, then no accepting transition can be taken, ensuring the LTL specification holds.

THEOREM 3.3. *Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a labelling function $\mathfrak{L} : \mathcal{X} \rightarrow 2^{\text{AP}}$ and an NBA $\mathcal{A} = (2^{\text{AP}}, \mathcal{Q}, q_0, \delta, \text{Acc})$ for $\neg\phi$. If for any $q_i \in \text{Acc}^{\text{start}}$, a transition safety certificate B_i can be found, then it can be guaranteed that $\mathfrak{S} \models_{\mathfrak{L}} \phi$.*

PROOF. By leveraging Lemma 3.2, all start states of accepting transitions are unreachable. Consequently, all accepting transitions never occur, ensuring that the system's language is not accepted and thereby guaranteeing $\mathfrak{S} \models_{\mathfrak{L}} \phi$. $\square$

3.2 Transition Persistence Certificate

In contrast, the *transition persistence certificate* offers an alternative approach. Instead of completely preventing the system from reaching states where accepting transitions may happen, this certificate proves that even if such states are reached, the corresponding accepting transitions can occur only a finite number of times. This is achieved by requiring a decrease in a certificate value each time an accepting transition is taken.

Definition 3.4 (transition persistence certificate). Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a labelling function $\mathfrak{L} : \mathcal{X} \rightarrow 2^{\text{AP}}$ and an NBA $\mathcal{A} = (2^{\text{AP}}, \mathcal{Q}, q_0, \delta, \text{Acc})$. For $(k, l) \in \text{Acc}^{\text{pair}}$, define the indicator function $I_{k,l}(x) = 1$ if $(q_k, \mathfrak{L}(x), q_l) \in \text{Acc}^{k,l}$, and 0 otherwise. A bounded function $B_{k,l}(x) : \mathcal{X} \rightarrow \mathbb{R}$ is a *transition persistence certificate* with respect to $\text{Acc}^{k,l}$ if there exists $\epsilon > 0$ such that for all $x_0 \in \mathcal{X}_0$, $x \in \mathcal{X}$, $x' = f(x)$, we have:

$$B_{k,l}(x_0) \geq 0, \quad (13)$$

$$B_{k,l}(x) \geq 0 \Rightarrow B_{k,l}(x') \geq 0, \quad (14)$$

$$B_{k,l}(x) \geq B_{k,l}(x') + I_{k,l}(x)\epsilon. \quad (15)$$

Intuitively, condition (13) guarantees that the certificate $B_{k,l}$ is non-negative at the initial state of the system; condition (14) ensures that $B_{k,l}$ remains non-negative along any trajectory of the system; condition (15) implies that $B_{k,l}$ decreases by at least ϵ whenever the state's label matches an accepting transition from q_k to q_l , and remains non-increasing otherwise.

Since accepting transitions from q_k to q_l can only occur when $(q_k, \mathfrak{L}(x), q_l) \in \text{Acc}^{k,l}$, the number of such transitions is bounded by $B_{k,l}(x_0)/\epsilon$, ensuring finite occurrence. This yields the following lemma. Its proof is provided in the Appendix A.

LEMMA 3.5. *If a transition persistence certificate $B_{k,l}$ can be found, then all runs of words in $\mathfrak{L}(\mathfrak{S})$ make accepting transitions from q_k to q_l happen finitely often.*

The next theorem is obtained by applying the lemma to every accepting transition pair.

THEOREM 3.6. *Given a system $\mathfrak{S} = (X, X_0, f)$, a labelling function $\mathfrak{L} : X \rightarrow 2^{AP}$ and an NBA $\mathcal{A} = (2^{AP}, Q, q_0, \delta, \text{Acc})$ for $\neg\phi$. if for any $(k, l) \in \text{Acc}^{\text{pair}}$, a corresponding transition persistence certificate $B_{k,l}$ can be found, then it can be guaranteed that $\mathfrak{S} \models_{\mathfrak{L}} \phi$.*

PROOF. By leveraging Lemma 3.5, all accepting transitions occur only finitely often, which ensures that the system's language is not accepted, thereby guaranteeing $\mathfrak{S} \models_{\mathfrak{L}} \phi$. $\square$

While both the methods established by Theorem 3.3 and Theorem 3.6 can be used to verify LTL properties, the two types of certificates are complementary and, when combined, significantly expand the range of systems that can be verified. In some scenarios, requiring the system to completely avoid certain states might be too strict, making transition safety certificates impossible to find. However, the system might still satisfy the property if the accepting transitions occur only finitely often, a condition that can be certified by a transition persistence certificate. Conversely, consider verifying $\text{G}\neg a$ using the NBA for $\text{F}a$, where $\text{Acc}^{0,0} = \{(q_0, \emptyset, q_0), (q_0, \{a\}, q_0)\}$. For the persistence certificate $B_{0,0}$, condition (15) requires $B_{0,0}(x) \geq B_{0,0}(f(x)) + \epsilon$ for any x where an accepting transition is enabled. This would demand an infinite descent in the bounded certificate value, which is impossible, making it impossible to satisfy the conditions for a transition persistence certificate, and thus such a certificate cannot exist. Yet, a transition safety certificate can successfully verify the property by proving that state q_0 (the start of the accepting transitions) is unreachable in the product system. Thus, providing both certificates covers a wider range of system behaviors, enhancing the method's applicability. We now present the main theorem for LTL verification. For each accepting transition start state in the NBA, we use either a transition safety certificate or transition persistence certificates to prove accepting transitions from that state occur finitely often.

In summary, the combination of transition safety certificates and transition persistence certificates offers a less conservative verification framework. It maintains a smaller search space compared to closure certificates while providing two distinct mechanisms to ensure that accepting transitions occur only finitely often, ultimately proving that the system satisfies its LTL specification.

4 Certificate Synthesis

This section develops automated synthesis techniques for transition certificates. We leverage the Counterexample-Guided Inductive Synthesis (CEGIS) framework [?], a well-established paradigm in barrier certificate synthesis, to iteratively refine candidate solutions. By alternating between candidate generation and formal verification, this approach ensures that the discovered certificates are valid and proves that the system meets the specified LTL properties. Given a system $\mathfrak{S} = (X, X_0, f)$ and an NBA $\mathcal{A} = (2^{AP}, Q, q_0, \delta, \text{Acc})$ representing $\neg\phi$, we adopt a uniform notation where B_k represents the transition safety certificate template for states $q_k \in \text{Acc}^{\text{start}}$, and $B_{k,l}$ represents the transition persistence certificate template for $\text{Acc}^{k,l}$. To find transition certificates, we present CEGIS frameworks for polynomial and neural network templates, respectively. The synthesis proceeds in two alternating phases: *candidate generation* and *verification*.

4.1 Polynomial Template CEGIS

4.1.1 *Transition Safety Certificates.* We define a parametric polynomial template:

$$B_k(x, q; \mathbf{c}) = \sum_{i=1}^m c_i p_i(x, q), \quad (16)$$

where $x \in \mathcal{X}$, $q \in \mathcal{Q}$, $\{p_1, \dots, p_m\}$ is a user-defined basis of monomials over $\mathcal{X} \times \mathcal{Q}$ with degree at most d , and $\mathbf{c} = (c_1, \dots, c_m) \in \mathbb{R}^m$ are unknown coefficients. Our goal is to find $\mathbf{c}$ such that $B_k(\cdot, \cdot; \mathbf{c})$ satisfies conditions (10), (11), and (12).

Candidate Generation. Based on certificate conditions, we encode the constraint for all points of the sampled dataset. For transition safety certificates, the constraint is:

$$\bigwedge_{(x,q) \in D_0} B_k(x, q; \mathbf{c}) \geq 0 \quad (17)$$

$$\wedge \bigwedge_{(x,q) \in D_u} B_k(x, q; \mathbf{c}) < 0 \quad (18)$$

$$\wedge \bigwedge_{\substack{(x,q) \in D_X \\ (q, \mathfrak{Q}(x), q') \in \delta}} [B_k(x, q; \mathbf{c}) \geq 0 \Rightarrow B_k(f(x), q'; \mathbf{c}) \geq 0], \quad (19)$$

where D_0, D_u, D_X are the datasets sampled from $\mathcal{X}_0 \times \{q_0\}$, $\mathcal{X} \times \{q_k\}$, and $\mathcal{X} \times \mathcal{Q}$, respectively. The coefficient synthesis involves logical implications (disjunctive constraints) between linear inequalities. We employed Z3 [?] as an SMT solver to directly handle the logical structure. If the constraint is satisfiable with a solution $\hat{\mathbf{c}}$, define the candidate certificate as $\hat{B}_k = B_k(\cdot, \cdot; \hat{\mathbf{c}})$.

Verification. To verify $\hat{B}_k$ is valid, we formulate the negation of conditions and verify they are unsatisfiable. For transition safety certificates, we check:

$$x \in \mathcal{X}_0 \wedge \hat{B}_k(x, q_0) < 0, \quad (20)$$

$$x \in \mathcal{X} \wedge \hat{B}_k(x, q_k) \geq 0, \quad (21)$$

$$x \in \mathcal{X} \wedge (q, \mathfrak{Q}(x), q') \in \delta \wedge \hat{B}_k(x, q) \geq 0 \wedge \hat{B}_k(f(x), q') < 0. \quad (22)$$

We choose dReal [?] as our SMT verifier to check these constraints. If the constraints are satisfiable (i.e., counterexamples exist), we add them to D_0, D_u , or D_X accordingly. The synthesis process then iterates until no counterexamples are found, yielding a valid transition safety certificate.

4.1.2 *Transition Persistence Certificates.* We define a parametric polynomial template:

$$B_{k,l}(x; \mathbf{c}) = \sum_{i=1}^m c_i p_i(x), \quad (23)$$

where $x \in \mathcal{X}$, $\{p_1, \dots, p_m\}$ are monomials over $\mathcal{X}$ and $\mathbf{c} \in \mathbb{R}^m$ are unknown coefficients. We seek $\mathbf{c}$ and $\epsilon > 0$ satisfying conditions (13), (14), and (15).

Candidate Generation. Based on certificate conditions, we encode the constraint for all points of the sampled dataset. For transition persistence certificates, the constraint is:

$$\bigwedge_{x \in D_0} B_{k,l}(x; \mathbf{c}) \geq 0 \quad (24)$$

$$\wedge \bigwedge_{x \in D_X} [B_{k,l}(x; \mathbf{c}) \geq 0 \Rightarrow B_{k,l}(f(x); \mathbf{c}) \geq 0] \quad (25)$$

$$\wedge \bigwedge_{\substack{x \in D_X \\ (q_k, \mathfrak{Q}(x), q_l) \notin Acc^{k,l}}} B_{k,l}(x; \mathbf{c}) \geq B_{k,l}(f(x); \mathbf{c}) \quad (26)$$

$$\wedge \bigwedge_{\substack{x \in D_X \\ (q_k, \mathfrak{Q}(x), q_l) \in Acc^{k,l}}} B_{k,l}(x; \mathbf{c}) \geq B_{k,l}(f(x); \mathbf{c}) + \epsilon, \quad (27)$$

where D_0, D_X are the datasets sampled from $\mathcal{X}_0$ and $\mathcal{X}$, respectively. We solve the coefficient synthesis problem using the Z3 solver [?]. If satisfiable with solution $(\hat{\mathbf{c}}, \hat{\epsilon})$, define the candidate transition persistence certificate as $\hat{B}_{k,l} = B_{k,l}(\cdot; \hat{\mathbf{c}})$.

Verification. To verify $\hat{B}_{k,l}$ is valid, we encode constraints of the negation of conditions and verify they are unsatisfiable. For transition persistence certificates, we check:

$$x \in \mathcal{X}_0 \wedge \hat{B}_{k,l}(x) < 0, \quad (28)$$

$$x \in \mathcal{X} \wedge \hat{B}_{k,l}(x) \geq 0 \wedge \hat{B}_{k,l}(f(x)) < 0, \quad (29)$$

$$x \in \mathcal{X} \wedge (q_k, \mathfrak{Q}(x), q_l) \in Acc^{k,l} \wedge \hat{B}_{k,l}(x) < \hat{B}_{k,l}(f(x)) + \hat{\epsilon}, \quad (30)$$

$$x \in \mathcal{X} \wedge (q_k, \mathfrak{Q}(x), q_l) \notin Acc^{k,l} \wedge \hat{B}_{k,l}(x) < \hat{B}_{k,l}(f(x)). \quad (31)$$

We use dReal [?] to check these constraints. If the constraints are satisfiable, we add them to D_0 or D_X accordingly. The synthesis process then iterates until no counterexamples are found, yielding a valid transition persistence certificate.

4.2 Neural Template CEGIS

Since neural networks have the capability to approximate arbitrary functions, using them as templates for certificates offers advantages over polynomials [?]. Assume that $x \in \mathbb{R}^n$ represents the n -dimensional input vector of the neural network, $L - 1$ represents the number of hidden layers, and y is the scalar output of the neural network with respect to the input x . Thus, the feedforward neural network can be expressed with the following structure:

$$\begin{cases} z_l = \mathbf{W}_l x_{l-1} + \mathbf{b}_l, & 0 < l \leq L \\ x_l = \sigma(z_l), & 0 < l \leq L \\ y = x_L \end{cases} \quad (5)$$

where,

- z_l denotes the output vector of the linear operation in the l -th layer;
- $\mathbf{W}_l$ and $\mathbf{b}_l$ denote the weight matrix and bias vector for the l -th layer of the neural network, respectively;
- σ denotes the activation function. Common activation functions include the Tanh function, Sigmoid function, ReLU function.

Candidate Generation. Based on certificate conditions, a loss function is designed for the training dataset. For transition safety certificates, the loss function is:

$$\begin{aligned}\mathcal{L}_B = & \sum_{(x,q) \in D_0} \text{ReLU}(-B_k(x, q; \theta)) \\ & + \sum_{(x,q) \in D_u} \text{ReLU}(B_k(x, q; \theta)) \\ & + \sum_{\substack{(x,q) \in D_X \\ (q, \mathfrak{Q}(x), q') \in \delta}} \text{ReLU}(B_k(x, q; \theta)) \cdot \text{ReLU}(-B_k(f(x), q'; \theta)),\end{aligned}\quad (32)$$

where D_0, D_u, D_X are datasets sampled from $\mathcal{X}_0 \times \{q_0\}$, $\mathcal{X} \times \{q_k\}$, and $\mathcal{X} \times Q$, respectively. The terms in the loss function offer intuitive measures of the extent to which the neural network violates the certificate conditions.

For transition persistence certificates, the loss function is:

$$\mathcal{L}_B = \sum_{\substack{x \in D_X \\ (q_k, \mathfrak{Q}(x), q_l) \in \text{Acc}^{k,l}}} \text{ReLU}(B_{k,l}(f(x); \theta) + \epsilon - B_{k,l}(x; \theta)) \quad (33)$$

$$+ \sum_{\substack{x \in D_X \\ (q_k, \mathfrak{Q}(x), q_l) \notin \text{Acc}^{k,l}}} \text{ReLU}(B_{k,l}(f(x); \theta) - B_{k,l}(x; \theta)), \quad (34)$$

where $D_X \subseteq \mathcal{X}$ is the dataset sampled from $\mathcal{X}$.

To generate a neural certificate candidate, we employ gradient descent techniques to minimize the loss function. When the loss decreases to 0, it indicates that the learned neural network can be a candidate certificate. In addition, the activation functions differ: the transition safety certificate employs the ReLU activation function, while the transition persistence certificate uses the squared activation function. This design eliminates the need to verify non-negativity conditions during verification. We define the candidate transition safety certificate as $\hat{B}_k = B_k(\cdot, \cdot; \hat{\theta})$ and the candidate transition persistence certificate to be $\hat{B}_{k,l} = B_{k,l}(\cdot; \hat{\theta})$.

Verification. To verify the candidate neural certificate is valid, we formulate the negation of conditions and use dReal [?] to verify they are unsatisfiable. For transition safety certificates, we check constraints (20), (21), and (22). For transition persistence certificates, we verify constraints (30) and (31). If counterexamples are found, they are added to the training set accordingly, and the synthesis process iterates until no counterexamples remain.

We choose dReal [?] as our SMT verifier, which ensures the correctness of its unsat decisions. Therefore, when dReal returns unsat for the given formula, it confirms that no solution exists within the specified precision, and the candidate certificate is valid. Otherwise, it means that dReal has found a counterexample that violates the certificate conditions. These counterexamples are added to the sample sets and the candidate generation phase is repeated. The process iterates until all verification queries are unsatisfiable (within the δ -completeness threshold of dReal), yielding a valid certificate.

5 Experimental Evaluation

We evaluate the proposed transition certificate synthesis methods on two benchmark systems to demonstrate: their effectiveness in formal LTL verification, and their computational efficiency compared to (neural) closure certificate methods. For the first benchmark, the Kuramoto Oscillator model (covering both 1D and 2D scenarios), we verify the system against specified LTL properties. For the second benchmark, the two-room temperature model, we demonstrate the versatility of our approach by synthesizing transition certificates to satisfy complex LTL requirements and highlight their complementary advantages in handling diverse temporal specifications. All experiments are conducted on Ubuntu 22.04.3 LTS (Intel i7-9750H, 8 GB RAM). During candidate generation,

Z3 addresses the constrained optimization for polynomial templates, whereas neural network templates are trained using the Adam optimizer (learning rate 0.001, 1000 iterations) to minimize the loss function. For formal verification, we employ dReal (precision 0.0001) to handle the transcendental sin functions in the Kuramoto Oscillator model, while Z3 is utilized for the temperature model.

5.1 Kuramoto Oscillator

5.1.1 One-Dimensional System. The first benchmark is a Kuramoto oscillator system, formally defined as $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ with dynamics adapted from [?]. The system operates within the state space $\mathcal{X} = [0, 2\pi]$, starting from an initial region $\mathcal{X}_0 = [\frac{4\pi}{9}, \frac{5\pi}{9}]$. To evaluate LTL verification, we specify an unsafe region $\mathcal{X}_u = [\frac{7\pi}{9}, \frac{8\pi}{9}]$ within the temporal properties. The discrete-time transition function f is given by:

$$f(x) = x + t_s \Omega + t_s K \sin(-x) - 0.532x^2 + 1.69, \quad (35)$$

where x represents the oscillator phase. The system parameters are set as follows: sampling time $t_s = 0.1$, natural frequency $\Omega = 0.01$, and coupling strength $K = 0.0006$.

We consider the LTL specification $\phi = \mathbf{G}\neg a$ over the atomic proposition set $AP = \{a\}$, with the labeling function defined as $\mathfrak{L}(x) = \{a\}$ for $x \in \mathcal{X}_u$ and $\mathfrak{L}(x) = \emptyset$ otherwise. Across the five implementations PT, ST, CC, NCC, and NNT, we use a unified semantic target: trajectories starting from $\mathcal{X}_0$ must avoid $\mathcal{X}_u$ forever. For PT and NNT, we use the transition-based negated automaton for $\neg\phi = \mathbf{F}a$, shown in Figure 1. CC and NCC use the corresponding state-based automaton for $\mathbf{F}a$. ST uses the strengthened state-based encoding $i \wedge \mathbf{F}u$, where i labels $\mathcal{X}_0$ and u labels $\mathcal{X}_u$, so that the accepting paths contain the state triplets required by the path-blocking construction; over trajectories generated from $\mathcal{X}_0$, this encoding is equivalent to $\mathbf{F}a$.

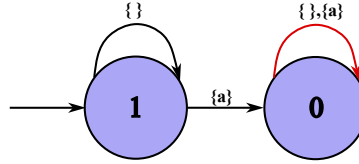


Fig. 1. NBA representing the negated specification $\mathbf{F}a$. Accepting transitions (highlighted in red) are depicted as self-loops at state q_0 .

Based on the definition of transition safety certificates, we implement a polynomial-based CEGIS framework to synthesize certificates that guarantee the satisfaction of the specified LTL property. The polynomial template is structured as follows:

$$B_0(x, q; \mathbf{c}) = \sum_{i=0}^4 c_i x^i + \sum_{j=1}^3 c_{4+j} q^j + c_8 x q.$$

The synthesis process begins by collecting 30 samples from region D_0 , 600 from D_u , and 1200 from $D_{\mathcal{X}}$. The candidate generation phase is formulated as a constraint satisfaction problem and solved using the Z3 solver. Given that the system dynamics involve the transcendental sin function, which lies beyond the decision procedures of Z3, we employ the dReal solver during the verification phase to identify counterexamples. The complete CEGIS loop converges in 1.76 seconds, successfully yielding a valid transition certificate:

$$B_0(x, q) = -1 + 2q^2 - \frac{7}{16}xq.$$

In addition to polynomial templates, we synthesize a neural transition certificate $B_0(x, q; \theta)$ using the CEGIS framework. The certificate is parameterized by a feedforward neural network with a 2-10-1 architecture, featuring

two input nodes for the pair (x, q) , a hidden layer of 10 neurons with ReLU activation, and a single linear output. The training process is initialized with a dataset of 250 samples (40 from D_0 , 70 from D_u , and 140 from D_X). During each iteration, the dReal solver is employed to identify any points violating the certificate conditions. These identified counterexamples are then incorporated into the training set to progressively refine the candidate certificate. The complete CEGIS loop converges in 6.81 seconds. The valid certificates obtained during the process are provided in the appendix.

5.1.2 Two-Dimensional System. The benchmark is further extended to a two-dimensional scenario with state space $\mathcal{X} = [0, \frac{8\pi}{9}]^2$ and initial region $\mathcal{X}_0 = [0, \frac{\pi}{9}]^2$. In this case, the unsafe region $\mathcal{X}_u$ is defined as the boundary corridors of the state space:

$$\mathcal{X}_u = ([0, \frac{8\pi}{9}] \times [\frac{5\pi}{6}, \frac{8\pi}{9}]) \cup ([\frac{5\pi}{6}, \frac{8\pi}{9}] \times [0, \frac{8\pi}{9}]). \quad (36)$$

The coupled system dynamics f are governed by:

$$f(x_1, x_2) = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + t_s \begin{bmatrix} \Omega + 1.69 \\ \Omega + 1.69 \end{bmatrix} + K t_s \begin{bmatrix} \sin(x_2 - x_1) \\ \sin(x_1 - x_2) \end{bmatrix} - 0.532 t_s \begin{bmatrix} x_1^2 \\ x_2^2 \end{bmatrix}. \quad (37)$$

To maintain consistency with the 1D analysis, we define the labeling function such that $\mathcal{Q}(x_1, x_2) = \{a\}$ if $(x_1, x_2) \in \mathcal{X}_u$ and $\emptyset$ otherwise. PT and NNT use the transition-based negated automaton for Fa, CC and NCC use the state-based automaton for Fa, and ST uses the strengthened state-based encoding $i \wedge Fu$, with i denoting $\mathcal{X}_0$ and u denoting $\mathcal{X}_u$. These encodings coincide on all trajectories generated from $\mathcal{X}_0$. The verification is therefore performed against the same negated behavior as in the 1D case.

For the two-dimensional system, we first employ a polynomial template $B_0(x_1, x_2, q; c)$. The synthesis process utilizes a dataset of 30 samples from D_0 , 729 from D_u , and 1,458 from D_X . Within 1.72 seconds, the CEGIS loop yields a valid transition certificate:

$$B_0(x_1, x_2, q) = -1 + \frac{1}{8}x_1x_2 + \frac{3}{2}q^2 - \frac{3}{32}qx_1^2 - \frac{153}{512}x_2q. \quad (38)$$

Furthermore, we implement a neural network template with a 3-15-1 architecture, featuring three input nodes for the tuple (x_1, x_2, q) and a hidden layer of 15 neurons with ReLU activation. To demonstrate the efficiency of our framework, we initialize the training with a set of points: 40 from D_0 , 90 from D_u , and 180 from D_X . The CEGIS loop converges in 110.95 seconds, successfully producing a valid neural certificate in appendix.

5.2 Two-Room Temperature

The second benchmark focuses on the formal verification of an interconnected temperature control system, denoted by $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, with its dynamical model adapted from [?]. The system operates within a state space $\mathcal{X} = [20, 34]^2$, originating from the initial configuration $\mathcal{X}_0 = [21, 24]^2$. The underlying LTL specification pertains to the target region $\mathcal{X}_{VF} = [20, 26]^2$. Specifically, the evolution of the system state is governed by the following discrete-time dynamics f :

$$f(x_1, x_2) = \begin{bmatrix} 1 - 2\alpha - \theta - \mu u(x_1) & \alpha \\ \alpha & 1 - 2\alpha - \theta - \mu u(x_2) \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \mu T_h \begin{bmatrix} u(x_1) \\ u(x_2) \end{bmatrix} + \theta \begin{bmatrix} T_e \\ T_e \end{bmatrix}, \quad (39)$$

where the constant parameters are assigned as $\alpha = 0.004$, $\theta = 0.01$, $\mu = 0.15$, $T_h = 40$, and $T_e = 0$. The state-dependent control term is defined as $u(x_i) = 0.59 - 0.011x_i$ for $i \in \{1, 2\}$.

Consider the set of atomic propositions $AP = \{a, b\}$, where the labeling function is defined as $\mathcal{Q}(x_1, x_2) = \{a\}$ for $(x_1, x_2) \in \mathcal{X}_0$, and $b \in \mathcal{Q}(x_1, x_2)$ for $(x_1, x_2) \in \mathcal{X}_{VF}$. The system is required to satisfy the LTL specification $\varphi = a \Rightarrow \text{FG}\neg b$. For all five implementations PT, ST, CC, NCC, and NNT, we use the same negated semantic target $\neg\varphi = a \wedge \text{GFB}$. The corresponding NBA is illustrated in Figure 2. To demonstrate the complementary roles

of transition safety certificates and transition persistence certificates, we synthesize both types of certificates for this benchmark.

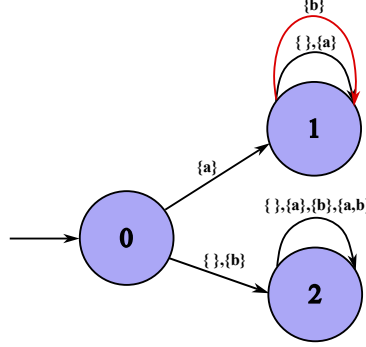


Fig. 2. NBA for $a \wedge GFb$ (negation of $a \Rightarrow FG\neg b$). Accepting transitions (red) are $(q_1, \{b\}, q_1)$ and $(q_1, \{a, b\}, q_1)$.

Consider the transition safety certificate template and transition persistence certificate template (seen in Appendix B). We simultaneously synthesize them. For the transition safety certificate, we collect 16 samples for D_0 , 196 for D_u , 588 for D_X . For the transition persistence certificate, we collect 16 samples for D_0 and 196 for D_X . We obtain a valid transition persistence certificate

$$B_{1,1}(x_1, x_2) = 27I_{[20,26]^2}(x_1, x_2) - x_1 I_{[20,26]^2}(x_1, x_2)$$

and $\epsilon = 0.25$ in 2.68 seconds.

We synthesize the transition safety certificate $B_1(x_1, x_2, q)$ for q_1 and the transition persistence certificate $B_{1,1}(x_1, x_2)$ for $Acc^{1,1}$. We synthesize $B_1(x_1, x_2, q)$ using the CEGIS with neural network template of architecture 3-10-1 (3 inputs for (x_1, x_2, q) , 10 hidden neurons with ReLU activation, 1 output). We synthesize $B_{1,1}(x_1, x_2)$ using the CEGIS with neural network template of architecture 2-3-1 (2 inputs for (x_1, x_2) , 3 hidden neurons with squared activation, 1 output). For the transition persistence certificate, we collect 100 samples for D_X . For the transition safety certificate, we collect 4 samples for D_0 , 9 for D_u , 18 for D_X . The entire CEGIS loop terminates in 1461.91 seconds and the valid certificates obtained are provided in the appendix..

5.3 Comparison

We primarily compare our approach with closure certificates [?] and neural closure certificates [?], as they represent recent developments in abstraction-free LTL verification. We also report local runs of the state-triplet method [?] as a foundational conservative baseline. The state-triplet method is more restrictive because it attempts to block all simple paths to accepting states, while closure certificates [?] were specifically developed to provide more general verification conditions than the state-triplet method. The core idea of closure certificates lies in characterizing an over-approximation of the transition closure, so closure certificates for LTL verification are defined over $X \times X \times Q \times Q$. In contrast, our method employs transition safety certificates to directly filter out unreachable automaton states or uses transition persistence certificates to prove that the accepting transitions occur finitely often, so the search space of transition certificates is at most $X \times Q \times Q$. This allows us to reduce the search space from $X \times X \times Q \times Q$ to $X \times Q \times Q$ and improves synthesis efficiency. For neural templates, we adopt the CEGIS framework, whereas the neural closure certificate approach relies on sampling and posteriori verification: after sampling the state space with a given density ϵ and safety margin η , the method trains until the loss converges to zero and then performs a posteriori verification to determine whether the synthesized certificate is valid.

Table 1. Synthesis time comparison (seconds)

Benchmark	Transition Certificates	State Triplet [?]	Closure Certificate [?]
Kuramoto 1D	1.76	0.10	600
Kuramoto 2D	1.72	1.13	6600
Two-Room Temperature	2.68	Timeout	5400

Table 2. Synthesis time comparison (seconds)

Benchmark	Transition Certificates	Neural Closure Certificate (local run)
Kuramoto 1D	6.81	Timeout
Kuramoto 2D	110.95	Timeout
Two-Room Temperature	1461.91	Timeout

We conduct two groups of comparisons: (1) polynomial-template CEGIS versus closure certificates, and (2) neural-template CEGIS versus neural closure certificates. The detailed experimental configurations are provided in Appendix B. Table 1 reports the synthesis times stated in the closure-certificate paper [?] for the three benchmarks and our local state-triplet runs; “Timeout” means that no formally verified state-triplet barrier was obtained within the time limit under the reported template. Table 2 reports our local runs of the neural closure-certificate baseline; there, “Timeout” indicates that the synthesis exceeded one hour. As shown in Table 1, when suitable templates are selected, our polynomial-template CEGIS method completes synthesis within seconds, whereas the reported closure-certificate baseline requires around 10 minutes for Kuramoto 1D, 1 hour and 50 minutes for Kuramoto 2D, and 1.5 hours for the two-room temperature benchmark. The state-triplet baseline verifies the two Kuramoto cases but times out on the two-room case under the tested template, consistent with its path-blocking conservatism for persistence-style properties. Similarly, as shown in Table 2, the neural-network template CEGIS requires approximately 10–20 minutes, which is slower than the polynomial-template version but remains considerably faster than the neural closure-certificate baseline under our tested settings. These comparisons indicate substantial efficiency gains obtained from reducing the search space complexity.

6 Conclusion

In this paper, we have proposed transition certificates as an efficient framework for verifying Linear Temporal Logic (LTL) properties of discrete-time dynamical systems with continuous state spaces. Our approach leverages transition safety and persistence certificates to reduce conservatism compared to state-triplet methods and minimize search spaces relative to closure certificates, achieving significant speedups in LTL verification. We have developed automated synthesis methods based on polynomial and neural network templates, demonstrating practicality through case studies like the Kuramoto oscillator and two-room temperature model. Nonetheless, our method faces a few shortcomings that motivate future work. First, the synthesis process, particularly for neural network templates, can be computationally demanding for high-dimensional systems with highly nonlinear dynamics, due to the bottleneck in SMT solving. Second, the framework relies on the construction of NBA for LTL specifications, which involves exponential complexity and may limit scalability to complex formulas. Additionally, the current work focuses on discrete-time systems, leaving continuous-time systems as an open challenge.

A Proof

LEMMA A.1. *Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a trajectory $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$, a labelling function $\mathfrak{L} : \mathcal{X} \rightarrow \Sigma$ and an NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, \text{Acc})$, if a transition safety certificate $B_k(x, q)$ with respect to $q_k \in \text{Acc}^{\text{start}}$ can be found, then it can be guaranteed that all runs of all words in $\mathfrak{L}(\mathfrak{S})$ cannot visit the state q_k .*

PROOF. By Definition 3.1, we have $\mathcal{Y}_u = \{(x, q_k) \mid x \in \mathcal{X}\}$ as the unsafe set in the product system $\mathfrak{S} \times \mathcal{A}$. We verify that B_k satisfies the barrier certificate conditions (4, 5, 6) with respect to $\mathcal{Y}_u$:

- Condition (10) states $B_k(x_0, q_0) \geq 0$ for all $(x_0, q_0) \in \mathcal{Y}_0$, which corresponds to condition (4).
- Condition (11) states $B_k(x, q_k) < 0$ for all $(x, q_k) \in \mathcal{Y}_u$, which corresponds to condition (5).
- Condition (12) states $B_k(x, q) \geq 0 \Rightarrow B_k(x', q') \geq 0$ for all $(x', q') \in F(x, q)$, which corresponds to condition (6).

By Theorem (2.2), any trajectory of the product system $\mathfrak{S} \times \mathcal{A}$ cannot visit $\mathcal{Y}_u$.

Now we establish the connection to automaton runs. For any word $w = \mathfrak{L}(\tau) \in \mathfrak{L}(\mathfrak{S})$ where $\tau = \langle x_0, x_1, \dots \rangle$ is a trajectory of $\mathfrak{S}$, the corresponding run r of w on $\mathcal{A}$ is a sequence $r = \langle (r_0, w_0, r_1), (r_1, w_1, r_2), \dots \rangle$ where $r_0 = q_0$. The sequence of product states $\langle (x_0, r_0), (x_1, r_1), \dots \rangle$ forms a trajectory in $\mathfrak{S} \times \mathcal{A}$. Since this trajectory cannot visit $\mathcal{Y}_u = \{(x, q_k) \mid x \in \mathcal{X}\}$, we have $r_i \neq q_k$ for all i . Therefore, runs of all words in $\mathfrak{L}(\mathfrak{S})$ cannot visit state q_k . $\square$

LEMMA A.2. *Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, an NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, \text{Acc})$, and $(k, l) \in \text{Acc}^{\text{pair}}$, if a transition persistence certificate $B_{k,l}$ can be found, then it can be guaranteed that all runs of the words in $\mathfrak{L}(\mathfrak{S})$ make accepting transitions from q_k to q_l happen finitely often.*

PROOF. We define $\mathcal{X}_{k,l} = \{x \mid (q_k, \mathfrak{L}(x), q_l) \in \text{Acc}^{k,l}\}$ as the set of states where accepting transitions from q_k to q_l can occur.

Consider any trajectory $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$ and its corresponding word $w = \mathfrak{L}(\tau)$. Let r be a run of w on $\mathcal{A}$ starting from q_0 . An accepting transition from q_k to q_l can occur at step i only if $(q_k, \mathfrak{L}(x_i), q_l) \in \text{Acc}^{k,l}$, which is equivalent to $x_i \in \mathcal{X}_{k,l}$. Therefore, the number of accepting transitions from q_k to q_l is bounded by the number of times the trajectory visits $\mathcal{X}_{k,l}$.

By Definition 3.4, we have:

- $B_{k,l}(x_0) \geq 0$ by condition (13).
- $B_{k,l}(x_i) \geq 0$ for all i by condition (14), since $B_{k,l}$ remains non-negative along the trajectory.
- $B_{k,l}(x_i) \geq B_{k,l}(x_{i+1}) + I_{k,l}(x_i)\epsilon$ by condition (15), where $I_{k,l}(x_i) = 1$ if $x_i \in \mathcal{X}_{k,l}$ and 0 otherwise.

Suppose the trajectory visits $\mathcal{X}_{k,l}$ at indices $i_1, i_2, \dots, i_m$ where m is the number of times accepting transitions occur. At each visit, $B_{k,l}$ decreases by at least ϵ :

$$B_{k,l}(x_0) \geq B_{k,l}(x_{i_1}) + \epsilon \geq B_{k,l}(x_{i_2}) + 2\epsilon \geq \dots \geq B_{k,l}(x_{i_m}) + m\epsilon. \quad (40)$$

Since $B_{k,l}(x_{i_m}) \geq 0$ by condition (14), we have $B_{k,l}(x_0) \geq m\epsilon$, which gives $m \leq \frac{B_{k,l}(x_0)}{\epsilon}$. Therefore, the number of accepting transitions from q_k to q_l is bounded, ensuring they occur only finitely often. $\square$

B Experiment

B.1 Template for Two-Dimensional Kuramoto Oscillator System

The transition safety certificate template for the two-dimensional Kuramoto Oscillator system is given by:

$$\begin{aligned} B_0(x_1, x_2, q) = & c_0 + c_1 \cdot x_1 + c_2 \cdot x_2 + c_3 \cdot q \\ & + c_4 \cdot I_{X_u}(x_1, x_2) + c_5 \cdot x_1 q + c_6 \cdot x_2 q \end{aligned}$$

B.2 Template for Two-Room Temperature System

For the *transition safety certificate*:

$$\begin{aligned} B_1(x_1, x_2, q) = & c_0 + c_1 \cdot I_{X_0}(x_1, x_2) + c_2 \cdot I_{X_{VF}}(x_1, x_2) \\ & + c_3 \cdot x_1 I_{X_0}(x_1, x_2) + c_4 \cdot x_2 I_{X_0}(x_1, x_2) \\ & + c_5 \cdot x_1 I_{X_{VF}}(x_1, x_2) + c_6 \cdot x_2 I_{X_{VF}}(x_1, x_2) \\ & + c_7 \cdot \max(x_1, x_2) + c_8 \cdot x_1^2 + c_9 \cdot x_2^2 + c_{10} \cdot q + c_{11} \cdot q^2. \end{aligned}$$

The template for the *transition persistence certificate* is as follows:

$$\begin{aligned} B_{1,1}(x_1, x_2) = & c_0 + c_1 \cdot I_{X_0}(x_1, x_2) + c_2 \cdot I_{X_{VF}}(x_1, x_2) \\ & + c_3 \cdot x_1 I_{X_0}(x_1, x_2) + c_4 \cdot x_2 I_{X_0}(x_1, x_2) \\ & + c_5 \cdot x_1 I_{X_{VF}}(x_1, x_2) + c_6 \cdot x_2 I_{X_{VF}}(x_1, x_2) \\ & + c_7 \cdot \max(x_1, x_2) + c_8 \cdot x_1^2 + c_9 \cdot x_2^2. \end{aligned}$$

B.3 Neural Network Certificate Synthesis Results

This section provides the neural network certificates synthesized in the three case studies.

B.3.1 Kuramoto Oscillator: One-Dimensional System. Valid transition safety certificate network parameters:

- **Network architecture:** 2 -> 10 -> 1 (ReLU activation network)
- **Layer 1 weights** $W^{(1)} \in \mathbb{R}^{10 \times 2}$:

$$W^{(1)} = \begin{bmatrix} 0.35423541 & 0.03017079 \\ 0.40347892 & 0.15485710 \\ -0.33290595 & 0.20670444 \\ 0.05953831 & -0.57081002 \\ -0.77764583 & -0.30270880 \\ 0.28167149 & -0.56571352 \\ -0.00433345 & 0.27621469 \\ -0.11347809 & -0.37141919 \\ 0.30172166 & -0.14517429 \\ 0.00136681 & 0.32282710 \end{bmatrix}$$

- **Layer 1 bias** $b^{(1)} \in \mathbb{R}^{10}$:

$$b^{(1)} = \begin{bmatrix} -0.83490396 & 0.57956743 & -0.47568363 & -0.27932465 & -0.28890583 \\ 0.16922584 & -0.66955644 & -0.23689666 & 0.49750403 & 0.55335736 \end{bmatrix}^\top$$

- **Layer 2 weights** $W^{(2)} \in \mathbb{R}^{1 \times 10}$:

$$W^{(2)} = \begin{bmatrix} -1.36676764 & 0.13470453 & -0.01585477 & -0.04601728 & 0.22054167 \\ -0.15794340 & -0.09915265 & 0.00324699 & -0.02935893 & 0.18765648 \end{bmatrix}$$

- **Layer 2 bias** $b^{(2)} \in \mathbb{R}$:

$$b^{(2)} = -0.25243226$$

- **Analytical expression:**

$$B(x, q) = W^{(2)} \cdot \text{ReLU}(W^{(1)} \cdot [x, q]^\top + b^{(1)}) + b^{(2)}$$

B.3.2 Kuramoto Oscillator: Two-Dimensional System. Valid transition safety certificate network parameters:

- **Network architecture:** 3 -> 15 -> 1 (ReLU activation network)
- **Layer 1 weights** $W^{(1)} \in \mathbb{R}^{15 \times 3}$:

$$W^{(1)} = \begin{bmatrix} -0.01797801 & 0.64069444 & -0.03856681 \\ -0.57299656 & -0.14924462 & -0.04329325 \\ -0.56604266 & -0.25653100 & 0.10918397 \\ -0.12088224 & 0.62192768 & -0.95162183 \\ -0.54926884 & -0.45272782 & 0.26691911 \\ -0.35724673 & 0.10176215 & -0.26044405 \\ -0.62156856 & 0.45307916 & -0.63630158 \\ -0.50465399 & -0.00161430 & 0.04746520 \\ -0.21175091 & 0.12297605 & -0.52375120 \\ 0.60632193 & 0.14016706 & -2.34126782 \\ -0.07868959 & -0.15929390 & -0.10671024 \\ 0.68689883 & -0.58823287 & -0.65129077 \\ -0.75197023 & 0.13900414 & -0.01829747 \\ -0.55132180 & 0.17512728 & -0.04693245 \\ -0.18292713 & 0.10734314 & -0.20697621 \end{bmatrix}$$

- **Layer 1 bias** $b^{(1)} \in \mathbb{R}^{15}$:

$$b^{(1)} = \begin{bmatrix} -0.23332441 & -0.25264311 & -0.20304658 & 0.00498547 & -0.41519192 \\ -0.18106091 & -0.32431474 & 0.83221281 & -0.54517263 & 0.07074507 \\ -0.18164203 & 0.44940028 & 0.60892761 & -0.63835907 & 0.36881346 \end{bmatrix}^T$$

- **Layer 2 weights** $W^{(2)} \in \mathbb{R}^{1 \times 15}$:

$$W^{(2)} = \begin{bmatrix} -0.22106327 & 0.12717371 & -0.04997402 & -0.50930381 & -0.18251525 \\ 0.08938225 & 0.48234484 & -0.43327656 & -0.12058055 & 0.42470711 \\ 0.11804953 & -0.65379494 & 0.25955433 & 0.05265374 & 0.01196656 \end{bmatrix}$$

- **Layer 2 bias** $b^{(2)} \in \mathbb{R}$:

$$b^{(2)} = 0.36412391$$

- **Analytical expression:**

$$B(x_1, x_2, q) = W^{(2)} \cdot \text{ReLU}(W^{(1)} \cdot [x_1, x_2, q]^T + b^{(1)}) + b^{(2)}$$

B.3.3 Two-Room Temperature System. Valid transition persistence certificate network parameters:

- **Network architecture:** 2 -> 3 -> 1
- **Decrease constant** ϵ : 0.01 (fixed)
- **Layer 1 weights** $W^{(1)} \in \mathbb{R}^{3 \times 2}$:

$$W^{(1)} = \begin{bmatrix} -0.12892260 & 0.07935464 \\ 0.39971098 & -0.35251060 \\ -0.61775041 & 0.61788529 \end{bmatrix}$$

- **Layer 1 bias** $b^{(1)} \in \mathbb{R}^3$:

$$b^{(1)} = [1.44339669 \quad -1.41302800 \quad -0.01615771]^T$$

B.4 Comparative Method Setting

B.4.1 State-Triplet Method. The state-triplet baseline follows the accepting-state NBA path-blocking scheme of [?]. For each accepting state, all simple paths from the initial automaton state to the accepting state are enumerated. For a triplet (q_i, q_j, q_k) , let Y_0 be the union of labels on $q_i \rightarrow q_j$, Y_1 the union of labels on $q_j \rightarrow q_k$, and Y_m the labels on the self-loop $q_j \rightarrow q_j$ when present. We synthesize a barrier B satisfying

$$B(x) \leq 0 \text{ on } Y_0, \quad B(x) \geq 10^{-2} \text{ on } Y_1,$$

and the invariance condition

$$x \in (Y_0 \cup Y_m \cup Y_1) \setminus Y_1, \quad f(x) \in Y_0 \cup Y_m \cup Y_1, \quad B(x) \leq 0 \Rightarrow B(f(x)) \leq 0.$$

Candidate coefficients are synthesized by Z3 and formally checked by dReal for the Kuramoto systems and by Z3 for the two-room system.

For the 1D Kuramoto benchmark, the state-triplet template is

$$B(x) = c_0 + c_1x + c_2x^2 + c_3x^3 + c_4x^4.$$

The synthesized certificate is

$$B(x) = -4.5x^2 + 2x^3,$$

which blocks the accepting path $0 \rightarrow 1 \rightarrow 2$ with $Y_0 = \{i\}$, $Y_1 = \{u\} \cup \{i, u\}$, and $Y_m = \emptyset \cup \{i\}$. The direct accepting edge $0 \rightarrow 2$ is discharged because its label region $\{i, u\}$ is empty.

For the 2D Kuramoto benchmark, the tested state-triplet template is

$$B(x_1, x_2) = c_0 + c_1x_1 + c_2x_2 + c_3x_1x_2 + c_4x_1^2 + c_5x_2^2.$$

The direct accepting edge $0 \rightarrow 2$ is again discharged by the empty label region $\{i, u\}$. The path $0 \rightarrow 1 \rightarrow 2$ is blocked by the synthesized certificate

$$B(x_1, x_2) = -x_1 - x_2 - 3.125x_1x_2 + 2x_1^2 + 2x_2^2,$$

with $Y_0 = \{i\}$, $Y_1 = \{u\} \cup \{i, u\}$, and $Y_m = \emptyset \cup \{i\}$. This run used dReal precision 10^{-3} for the dynamics-dependent invariance check.

For the two-room benchmark, the tested state-triplet template is

$$B(x_1, x_2) = c_0 + c_1x_1 + c_2x_2 + c_3x_1x_2 + c_4x_1^2 + c_5x_2^2 + c_6 \max(x_1, x_2).$$

The accepting path $0 \rightarrow 2 \rightarrow 1$, with $Y_0 = \{a\} \cup \{a, b\}$, $Y_1 = \{b\} \cup \{a, b\}$, and $Y_m = \emptyset \cup \{a\}$, is not discharged before timeout. This is expected for this persistence-style benchmark: since $X_0 \subseteq X_{VF}$, traces from the initial region can satisfy both a and b , so the accepting state of the state-based NBA is reachable and the path-blocking ST obligation is too conservative.

Table 3. State-triplet path-blocking results

Benchmark	Blocked/resolved path	Status	Time (s)
1D Kuramoto	$0 \rightarrow 1 \rightarrow 2$; $0 \rightarrow 2$ empty	Proved	0.095
2D Kuramoto	$0 \rightarrow 1 \rightarrow 2$; $0 \rightarrow 2$ empty	Proved	1.13
Two-Room	$0 \rightarrow 2 \rightarrow 1$ unresolved	Timeout	Timeout

B.4.2 Closure Certificates Method. The synthesis process uses Z3 to find candidate certificates and dReal to find counterexamples, with a dReal precision of 0.0001. For the Kuramoto benchmarks, CC uses the original state-based automaton for the standard negated safety objective Fa . The strengthened encoding $i \wedge Fu$ is used only by ST, where state triplets are required. For the two-room benchmark, CC uses the negated objective $a \wedge GFb$.

One-Dimensional Kuramoto Oscillator System: The NBA is defined as $(\Sigma, Q, q_0, \delta, \text{Acc})$ to represent Fa .

$$\begin{aligned} \text{Acc} &= \{0\} \\ Q &= \{1, 0\} \\ q_0 &= 1 \\ \Sigma &= \{\emptyset, \{a\}\} \\ \delta &= \{(1, \emptyset, 1), (1, \{a\}, 0), (0, \emptyset, 0), (0, \{a\}, 0)\} \\ a &\text{ is defined as } x \in X_u \end{aligned}$$

The certificate templates are:

$$\begin{aligned} T_{0,0} &= c_0 + c_1x + c_2y \\ T_{0,1} &= c_3 + c_4x + c_5y \\ T_{1,0} &= c_6 + c_7x + c_8y \\ T_{1,1} &= c_9 + c_{10}x + c_{11}y \end{aligned}$$

The final synthesized coefficients are shown in Table 4.

Table 4. Coefficients for the synthesized closure certificates (One-Dimensional System)

Template	c_0	c_1	c_2
$T_{0,0}(x, y)$	0.000000	0.000000	0.000000
$T_{0,1}(x, y)$	-0.500000	0.000000	0.000000
$T_{1,0}(x, y)$	-4.310550	1.764117	0.000000
$T_{1,1}(x, y)$	0.000000	0.000000	0.000000

Two-Dimensional Kuramoto Oscillator System: Timeout.

Two-Room Temperature System: Timeout.

B.4.3 Neural Closure Certificates Method. The configuration for the neural closure certificates is summarized in Table 5. A sampling density ξ of 0.01 and a safety margin η of 0.01 are used for all systems. The Lipschitz constant is estimated using the spectral norm-based algorithm. For the Kuramoto benchmarks, NCC uses the original state-based automaton for Fa ; for the two-room benchmark, NCC uses $a \wedge GFb$.

Table 5. Neural network architecture and parameters for Neural Closure Certificates

System	Network Architecture	Activation	Lipschitz Estimation
1D Kuramoto Oscillator	(4-80-1)	ReLU	Spectral Norm
2D Kuramoto Oscillator	(6-80-1)	ReLU	Spectral Norm
Two-Room Temperature	(6-80-1)	ReLU	Spectral Norm

Temporary page!

L^AT_EX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because L^AT_EX now knows how many pages to expect for this document.